PRODUCT REQUIREMENTS DOCUMENT

AI Dashboard Generator

Next.js Frontend + Golang Backend

Production Company Grade PRD

Blueprint untuk membangun platform generator dashboard berbasis prompt, controlled schema, component registry, preview renderer, versioning, dan credit billing.

Document Field	Value
Product Name	DashboardCraft AI / nama final dapat diganti
Document Type	Product Requirements Document
Target Platform	Web SaaS
Frontend	Next.js App Router, TypeScript, Tailwind CSS, shadcn/ui
Backend	Golang API, PostgreSQL, Redis, Queue Worker
AI Strategy	Schema-first controlled generation, bukan AI free-form code generation
Prepared For	Rivael Manurung
Version	v1.0
Status	Ready for Engineering Planning

Product Principle

AI tidak boleh bebas membuat UI dari nol. AI hanya boleh menghasilkan normalized intent dan JSON schema yang divalidasi. UI final dirender oleh component registry dan design system milik platform.

Daftar Isi

1. Executive Summary

Produk ini adalah SaaS AI Dashboard Generator yang memungkinkan user membuat halaman dashboard production-grade dari prompt. Output tidak dibangun secara bebas oleh AI, melainkan melalui pipeline terkontrol: prompt dianalisis menjadi intent, intent dikonversi menjadi dashboard schema, schema divalidasi, lalu direnderer menggunakan component registry dan design system internal.

Fokus utama produk adalah menghasilkan dashboard yang konsisten, responsif, mudah dicopy ke project developer, dan bisa disimpan dalam project multi-page. Sistem ditujukan untuk developer, freelancer, agency, founder SaaS, dan internal product team yang ingin mempercepat pembuatan admin dashboard, CRUD page, analytics page, form page, dan detail page.

Area	Keputusan Produk
Positioning	AI assistant untuk membuat dashboard UI production-ready, bukan generic website builder.
Primary Value	Mengurangi waktu pembuatan dashboard dari beberapa hari menjadi hitungan menit.
Output	Schema JSON, rendered preview, generated code, page version history.
Quality Strategy	Component registry, domain presets, design token, schema validation, linting, dan preview isolation.
Commercial Model	Credit-based generation dengan paket top-up atau subscription.

2. Product Vision & Goals

2.1 Vision

Menjadi platform AI dashboard generator yang menghasilkan UI konsisten, profesional, dan siap dipakai developer tanpa perlu memperbaiki layout acak dari AI.

2.2 Product Goals

- User dapat membuat project dan halaman dashboard dari prompt dalam satu flow yang jelas.
- Hasil dashboard harus konsisten antar halaman dalam project yang sama.
- AI output wajib valid terhadap schema; output invalid tidak boleh direnderer.
- User dapat melihat preview desktop, tablet, dan mobile.
- User dapat copy code atau export halaman.
- Sistem mendukung versioning agar user bisa rollback.
- Credit terpotong secara aman hanya untuk generation yang berhasil atau sesuai aturan produk.

2.3 Non-Goals MVP

- Tidak membuat backend aplikasi user secara otomatis.
- Tidak menjamin business logic user selesai.
- Tidak membiarkan AI generate arbitrary React code tanpa sandbox dan validator.
- Tidak mendukung plugin marketplace pada fase MVP.

3. Target Users & Use Cases

Persona	Masalah	Value yang Diberikan
Freelance Developer	Butuh membuat admin panel cepat untuk client.	Generate dashboard, list, form, dan detail page yang konsisten.
Startup Founder	Butuh prototype SaaS untuk validasi	Bisa membuat UI dashboard cepat

	ide.	tanpa designer full-time.
UI Engineer	Butuh starter layout yang rapi.	Copy code berbasis component system yang predictable.
Agency	Butuh banyak variasi dashboard untuk proposal.	Template dan prompt-to-page mempercepat delivery.
Internal Product Team	Butuh internal tools dan analytics page.	Generate struktur halaman berdasarkan domain preset.

3.1 Primary Use Cases

- Membuat dashboard admin rumah sakit, sekolah, inventory, finance, CRM, agriculture, donation, POS, dan HR.
- Membuat halaman CRUD list dengan table, filter, action, status badge, dan pagination placeholder.
- Membuat form page dengan section grouping, validation hint, upload placeholder, dan CTA.
- Membuat detail page dengan profile panel, activity timeline, tabs, dan related data.
- Merefine section tertentu tanpa merusak layout global seperti sidebar, topbar, dan theme.

4. Product Scope

Module	MVP	V1	V2
Auth & Workspace	Email login, project dashboard	Team workspace	Organization billing
Project Management	Create/read/update/delete project	Project settings	Role-based collaboration
AI Generation	Prompt to schema	Section refinement	Multi-step agent
Renderer	Preview from schema	Responsive preview	Interactive visual editor
Code Output	Copy TSX/HTML	Export page file	Export project zip
Templates	Free gallery	Premium themes	Marketplace
Billing	Credit ledger	Payment integration	Subscription + usage
Quality	Schema validation	Code lint/typecheck	Automated visual QA

5. End-to-End User Flow

21. User register/login.
22. User membuka Project Dashboard.
23. User membuat New Project dengan nama, deskripsi, domain, dan default theme.
24. User memilih Create Page.
25. User memilih page type: Dashboard, List, Form, Detail, Login, atau Analytics.
26. User menulis prompt kebutuhan halaman.
27. Backend membuat generation job dan mengecek credit.
28. AI menghasilkan normalized intent dan dashboard schema.
29. Schema divalidasi dengan schema validator.
30. Renderer menampilkan preview berdasarkan component registry.
31. User dapat refine section tertentu atau generate ulang.
32. User menyimpan versi halaman.
33. User melihat code, copy, atau export.

```
User Prompt
  -> Prompt Analyzer
  -> Intent Normalizer
  -> Schema Generator
```

```
  -> Schema Validator
  -> Component Renderer
  -> Preview Sandbox
  -> Save Page Version
  -> Code Output
```

6. AI Dashboard Generation System

6.1 Core Principle

AI hanya bertugas merencanakan struktur halaman. UI final harus dibuat oleh renderer internal. Ini mencegah hasil acak, spacing random, class Tailwind liar, warna tidak konsisten, dan code yang tidak maintainable.

6.2 Generation Pipeline

Stage	Responsibility	Output
Prompt Analyzer	Membaca intent user, domain, page type, complexity.	Normalized intent JSON
Domain Preset Resolver	Memilih preset domain seperti hospital, school, finance, inventory.	Preset config
Schema Generator	Membuat schema halaman sesuai allowed components.	Page schema JSON
Schema Validator	Validasi strict terhadap JSON schema.	Valid/invalid result
Quality Scorer	Menilai completeness, density, visual balance.	Quality score
Renderer	Render preview dari schema.	Preview page
Code Generator	Generate code dari schema dan template.	TSX/HTML output
Version Writer	Menyimpan schema, code, prompt, dan metadata.	Page version

6.3 Allowed Page Types

Page Type	Purpose	Required Sections
dashboard	Halaman ringkasan metric dan insight.	stats, chart, table/activity
list	CRUD list/table management.	toolbar, filters, table, pagination
form	Create/update data.	form sections, validation hints, action footer
detail	Profile/detail entity.	summary card, tabs, timeline/table
login	Auth landing/login.	auth form, brand panel
analytics	Advanced reporting.	stats, filters, charts, comparison table

6.4 Component Registry

```
Allowed components:
- statsGrid
- chartPanel
- dataTable
- activityTimeline
- filterToolbar
- formSection
```

```
- profileSummary
- tabbedContent
- kanbanBoard
- calendarView
- notificationList
- emptyState
- actionFooter
```

6.5 Schema Example

```
{
  "pageType": "dashboard",
  "domain": "hospital",
  "layout": "admin-sidebar",
  "theme": "medical-clean",
  "sections": [
    {
      "type": "statsGrid",
      "items": [
        { "label": "Total Patients", "value": "12,430", "trend": "+12%" },
        { "label": "Appointments Today", "value": "328", "trend": "+8%" },
        { "label": "Available Doctors", "value": "86", "trend": "+4%" },
        { "label": "Monthly Revenue", "value": "$128,400", "trend": "+18%" }
      ]
    },
    {
      "type": "chartPanel",
      "title": "Patient Visits",
      "chartType": "line"
    },
    {
      "type": "dataTable",
      "title": "Recent Appointments",
      "columns": ["Patient", "Doctor", "Department", "Time", "Status"]
    }
  ]
}
```

7. Feature Requirements

ID	Feature	Requirement	Priority
F-001	Authentication	User bisa register, login, logout, reset password.	Must
F-002	Project Dashboard	User bisa melihat daftar project, search, filter, dan create project.	Must
F-003	Create Project	User memasukkan nama, deskripsi, domain, default theme.	Must
F-004	Create Page	User memilih page type dan menulis prompt.	Must
F-005	AI Generate Schema	Backend generate schema dari prompt dan domain preset.	Must
F-006	Schema Validation	Invalid schema tidak boleh disimpan sebagai versi aktif.	Must
F-007	Preview Renderer	User melihat hasil render halaman.	Must

F-008	Responsive Preview	User bisa switch desktop/tablet/mobile.	Must
F-009	Code Viewer	User bisa melihat TSX/HTML output.	Must
F-010	Copy Code	User bisa copy code ke clipboard.	Must
F-011	Page Versioning	Setiap generate/refine membuat version baru.	Must
F-012	Section Refinement	User bisa mengubah section tertentu tanpa regenerate semua.	Should
F-013	Template Gallery	User bisa browse free templates.	Should
F-014	Theme System	User bisa pilih theme dan premium theme.	Should
F-015	Credit Billing	Credit berkurang saat generation berhasil.	Must
F-016	Export	User bisa export page atau project.	Could

8. Frontend Requirements - Next.js

8.1 Recommended Stack

Layer	Technology	Reason
Framework	Next.js App Router + TypeScript	Routing modern, server/client component separation, production-friendly.
UI	Tailwind CSS + shadcn/ui	Consistent design system dan fast iteration.
Form	React Hook Form + Zod	Strong validation dan predictable form state.
State	TanStack Query + Zustand	Server state dan UI state dipisah.
Editor	Monaco Editor	Code viewer/editor professional.
Preview	Sandboxed iframe	Isolasi generated preview dari main app.
Charts	Recharts / Tremor-compatible chart layer	Dashboard charts reusable.
Animation	Framer Motion	Micro-interaction tanpa overdesign.

8.2 Frontend Routes

Route	Page	Access
/	Landing Page	Public
/login	Login	Public
/register	Register	Public
/pricing	Pricing	Public

/freebies	Template Gallery	Public
/app	Project Dashboard	Authenticated
/app/projects/new	Create Project	Authenticated
/app/projects/[id]	Project Detail	Authenticated
/app/projects/[id]/pages/[pageId]	Builder Workspace	Authenticated
/app/billing	Credit & Billing	Authenticated
/app/settings	User Settings	Authenticated
/admin	Admin Console	Admin

8.3 Builder Workspace Layout

```
Builder Workspace
- Left Panel: Project pages, theme selector, page settings
- Center Panel: Responsive preview iframe
- Right Panel: Prompt/refinement panel, component tree, code tabs
- Top Bar: Save, version selector, copy code, export, credit status
```

8.4 Frontend Quality Rules

- Tidak ada generated code yang dieksekusi langsung di main DOM tanpa sandbox.
- Preview harus berada dalam iframe dengan sandbox policy.
- Semua API request menggunakan typed client dan error boundary.
- Copy code harus mengambil code dari server version yang tersimpan, bukan state sementara.
- Page builder harus autosave draft prompt, tetapi schema final hanya disimpan setelah valid.

9. Backend Requirements - Golang

9.1 Recommended Stack

Layer	Technology	Reason
HTTP Framework	Gin	Mature, performant, middleware ecosystem stabil.
Database	PostgreSQL	JSONB support untuk schema dan ledger consistency.
Query	sqlc + pgx	Type-safe SQL tanpa ORM berat.
Cache/Queue	Redis	Rate limit, job queue, idempotency lock.
Auth	JWT access + refresh token rotation	Secure SaaS auth flow.
Worker	Golang worker process	AI generation async dan retryable.
Validation	go-playground/validator + JSON Schema validation	Input dan schema strict.
Observability	OpenTelemetry + structured logs	Trace generation job dan API latency.

9.2 Backend Services

Service	Responsibility
AuthService	Register, login, refresh, logout, password reset.
ProjectService	CRUD project dan ownership check.

PageService	CRUD pages, active version, page metadata.
GenerationService	Create generation job, call AI provider, validate schema, save version.
ThemeService	Manage themes dan tokens.
TemplateService	Free/premium template gallery.
CreditService	Credit balance, ledger, reserve, consume, refund.
RendererService	Generate code dari schema dan component templates.
AuditService	Record critical product events.

9.3 Backend Architecture

```
cmd/api/main.go
cmd/worker/main.go
internal/http/handlers
internal/http/middleware
internal/services
internal/repositories
internal/domain
internal/ai
internal/schema
internal/renderer
internal/billing
internal/platform/postgres
internal/platform/redis
internal/platform/logger
```

9.4 Critical Backend Rules

- Semua mutation harus memverifikasi ownership user terhadap project/page.
- Credit deduction harus atomic menggunakan database transaction.
- Generation job harus idempotent menggunakan request_id.
- AI response tidak boleh dipercaya sebelum schema validation.
- Failed generation harus memiliki status dan retry policy yang jelas.
- Refresh token harus disimpan hashed dan rotated.
- Admin endpoint wajib RBAC, bukan sekadar hidden UI.

10. Database & Data Model

Table	Purpose
users	Data user dan auth identity.
refresh_tokens	Refresh token rotation dan revoke.
projects	Workspace project milik user.
project_pages	Halaman dalam project.
page_versions	Riwayat schema/code/prompt per halaman.
themes	Theme tokens dan metadata.
templates	Template gallery free/premium.
generation_jobs	Async AI job tracking.
credit_wallets	Saldo credit user.
credit_transactions	Ledger credit immutable.

audit_logs	Security dan product event log.

10.1 Core SQL Draft

```
CREATE TABLE users (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    name VARCHAR(120) NOT NULL,
    email VARCHAR(190) NOT NULL UNIQUE,
    password_hash TEXT NOT NULL,
    role VARCHAR(30) NOT NULL DEFAULT 'user',
    created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
    updated_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE TABLE projects (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    user_id UUID NOT NULL REFERENCES users(id),
    name VARCHAR(160) NOT NULL,
    description TEXT,
    domain VARCHAR(80),
    default_theme_slug VARCHAR(100),
    created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
    updated_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE TABLE project_pages (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    project_id UUID NOT NULL REFERENCES projects(id) ON DELETE CASCADE,
    name VARCHAR(160) NOT NULL,
    slug VARCHAR(180) NOT NULL,
    page_type VARCHAR(50) NOT NULL,
    current_version_id UUID,
    created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
    updated_at TIMESTAMPTZ NOT NULL DEFAULT now(),
    UNIQUE(project_id, slug)
);

CREATE TABLE page_versions (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    page_id UUID NOT NULL REFERENCES project_pages(id) ON DELETE CASCADE,
    version_number INT NOT NULL,
    prompt TEXT NOT NULL,
    schema_json JSONB NOT NULL,
    generated_code TEXT,
    quality_score NUMERIC(5,2),
    created_by UUID NOT NULL REFERENCES users(id),
    created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
    UNIQUE(page_id, version_number)
);

CREATE TABLE generation_jobs (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    user_id UUID NOT NULL REFERENCES users(id),
    project_id UUID NOT NULL REFERENCES projects(id),
    page_id UUID REFERENCES project_pages(id),
    request_id VARCHAR(120) NOT NULL UNIQUE,
    status VARCHAR(30) NOT NULL,
    prompt TEXT NOT NULL,
    error_message TEXT,
    credit_cost INT NOT NULL DEFAULT 1,
    started_at TIMESTAMPTZ,
    finished_at TIMESTAMPTZ,
    created_at TIMESTAMPTZ NOT NULL DEFAULT now()
```

```
);

CREATE TABLE credit_wallets (
    user_id UUID PRIMARY KEY REFERENCES users(id),
    balance INT NOT NULL DEFAULT 0,
    updated_at TIMESTAMPTZ NOT NULL DEFAULT now(),
    CHECK(balance >= 0)
);

CREATE TABLE credit_transactions (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    user_id UUID NOT NULL REFERENCES users(id),
    type VARCHAR(30) NOT NULL,
    amount INT NOT NULL,
    balance_after INT NOT NULL,
    reference_type VARCHAR(50),
    reference_id UUID,
    description TEXT,
    created_at TIMESTAMPTZ NOT NULL DEFAULT now()
);
```

11. API Contract

Method	Endpoint	Purpose	Auth
POST	/v1/auth/register	Register user	Public
POST	/v1/auth/login	Login user	Public
POST	/v1/auth/refresh	Rotate refresh token	Refresh
POST	/v1/auth/logout	Revoke token	User
GET	/v1/projects	List projects	User
POST	/v1/projects	Create project	User
GET	/v1/projects/:id	Project detail	Owner
POST	/v1/projects/:id/pages	Create page shell	Owner
GET	/v1/pages/:id	Get page detail	Owner
POST	/v1/pages/:id/generate	Generate or regenerate page	Owner + Credit
POST	/v1/pages/:id/refine	Refine selected section	Owner + Credit
GET	/v1/pages/:id/versions	List page versions	Owner
POST	/v1/pages/:id/ versions/:versionId/restore	Restore version	Owner
GET	/v1/themes	List themes	User/Public
GET	/v1/templates	List templates	Public
GET	/v1/billing/wallet	Credit wallet	User

11.1 Generate Request

```
POST /v1/pages/{pageId}/generate
Headers:
  Authorization: Bearer <access_token>
  Idempotency-Key: <uuid>

Body:
{
```

```
  "prompt": "Create a modern hospital dashboard for patients and appointments",
  "pageType": "dashboard",
  "themeSlug": "medical-clean",
  "domain": "hospital",
  "outputMode": "tsx"
}
```

11.2 Generate Response

```
{
  "jobId": "uuid",
  "status": "queued",
  "creditCost": 1,
  "message": "Generation job has been queued"
}
```

12. Security & Abuse Prevention

Risk	Required Control
Prompt injection	AI system prompt terpisah, schema validation, deny unsupported output.
Generated XSS	Generated preview disandbox, sanitize output, no arbitrary script execution.
Credit abuse	Rate limit, idempotency key, atomic ledger, anomaly detection.
Data leak antar user	Ownership check di repository/service layer.
Token theft	Short-lived access token, refresh rotation, hashed refresh token.
DoS generation	Queue limit, per-user concurrent job limit, max prompt length.
AI cost spike	Budget guard per user, max retries, provider timeout.
Admin abuse	Audit logs, RBAC, least privilege.

Security Decision

Generated page preview harus menggunakan iframe sandbox. Jangan pernah inject generated HTML/JS langsung ke DOM utama aplikasi builder.

13. Billing & Credit System

Sistem credit harus berbasis ledger, bukan sekadar integer balance yang dikurangi tanpa histori. Semua top-up, usage, refund, dan adjustment disimpan sebagai credit_transactions.

Transaction Type	Sign	Example
topup	+	User membeli 100 credit.
usage	-	User generate dashboard.
refund	+	Generation gagal karena error provider.
adjustment	+/-	Manual admin correction.

13.1 Credit Consumption Rules

- MVP: 1 successful page generation = 1 credit.
- Premium theme atau high-quality mode dapat menggunakan 2 credits.
- Credit dipotong saat job masuk processing menggunakan reserve/consume model.
- Jika AI provider gagal sebelum schema valid, credit direfund otomatis.
- Jika user cancel saat job sudah selesai, credit tidak direfund.

14. Non-Functional Requirements

Category	Requirement
Performance	API p95 < 300ms untuk non-generation endpoint. Generation async.
Scalability	Worker horizontal scalable; queue-based generation.
Reliability	Job retry maksimal 2 kali untuk transient provider error.
Availability	Target 99.5% untuk MVP production.
Data Integrity	Credit ledger immutable dan transaction-safe.
Maintainability	Backend modular service/repository; FE feature-based folder.
Observability	Trace setiap generation job dari request sampai output.
Compliance	Do not store raw payment card data. Gunakan payment gateway.

15. QA Strategy

Layer	Test Required
Unit Test	Schema validator, credit service, prompt normalizer, renderer mapper.
Integration Test	Auth, project CRUD, page generation lifecycle, credit transaction.
Contract Test	FE typed API client vs backend OpenAPI spec.
E2E Test	Login, create project, generate page, preview, copy code.
Security Test	Ownership bypass, prompt injection, XSS preview, rate limit.
Visual Test	Snapshot untuk component registry dan theme rendering.
Load Test	Queue throughput dan concurrent generation limit.

15.1 Definition of Done

- Endpoint memiliki validation, auth, ownership check, dan test.
- Setiap generation menghasilkan job record dan audit log.
- Schema invalid tidak pernah menjadi current active version.
- Credit transaction tercatat lengkap dan balance_after benar.
- Frontend memiliki loading, error, empty, dan success state.
- Preview responsive desktop/tablet/mobile berjalan tanpa layout break fatal.

16. Analytics & Observability

Metric	Purpose

generation_started	Mengukur usage awal.
generation_success_rate	Melihat kualitas AI pipeline.
schema_validation_fail_rate	Mengukur prompt/schema reliability.
average_quality_score	Mengukur kualitas output.
copy_code_clicked	Mengukur value real ke developer.
refine_used	Mengukur kebutuhan editing.
credit_conversion_rate	Mengukur monetization.
template_used	Menilai template mana yang paling laku.

17. Release Roadmap

Phase	Scope	Exit Criteria
Phase 0 - Foundation	Auth, project CRUD, schema model, component registry basic.	User bisa create project dan page kosong.
Phase 1 - MVP Generator	Prompt to schema, validation, preview, save version.	User bisa generate dashboard valid dan preview.
Phase 2 - Code Output	TSX/HTML generator, Monaco viewer, copy code.	User bisa copy code dari saved version.
Phase 3 - Credit Billing	Wallet, ledger, usage, top-up integration.	Credit usage aman dan auditable.
Phase 4 - Templates	Freebies gallery, template detail, clone to project.	Public template bisa dipakai untuk acquisition.
Phase 5 - Refinement	Section-level refine, version diff, restore.	User bisa refine tanpa merusak layout global.
Phase 6 - Production Hardening	Visual test, rate limit, observability, admin console.	Siap public beta.

18. Acceptance Criteria

- User baru dapat register, login, membuat project, dan membuat page.
- Prompt sederhana seperti "buat dashboard rumah sakit" menghasilkan schema valid.
- Dashboard memiliki minimal stats, chart, dan table/activity.
- Theme tetap konsisten setelah regenerate dan refine.
- Preview bisa dilihat dalam desktop, tablet, dan mobile.
- Generated code bisa dicopy dan sesuai dengan version yang sedang aktif.
- Credit berkurang hanya sesuai rule billing dan tercatat di ledger.
- User tidak bisa mengakses project/page milik user lain.
- Invalid AI response tidak menyebabkan page rusak.
- Admin bisa melihat job error dan audit event tanpa membuka data sensitif user secara berlebihan.

19. Risks & Mitigation

Risk	Impact	Mitigation
AI output ngaco	Preview/code buruk.	Schema-first generation, strict validator,

		domain presets.
Biaya AI membengkak	Margin turun.	Queue, timeout, retry limit, budget guard, model routing.
Generated code tidak usable	User churn.	Template-based code generator dan code linting.
Layout tidak konsisten multi-page	Produk terlihat amatir.	Project-level theme lock dan reusable layout schema.
Security XSS preview	Akun/data user terdampak.	Sandbox iframe dan sanitization.
Credit race condition	Saldo salah.	DB transaction, row lock, idempotency key.
Vendor lock-in AI	Sulit migrasi.	AI provider abstraction dan prompt versioning.

20. Engineering Notes

20.1 Folder Structure - Frontend

```
src/
  app/
    (public)/
    (auth)/
    app/
      projects/
      billing/
      settings/
  components/
    ui/
    builder/
    dashboard-registry/
  features/
    auth/
    projects/
    pages/
    generation/
    billing/
    templates/
  lib/
    api-client.ts
    auth.ts
    schema.ts
    constants.ts
  stores/
  types/
```

20.2 Folder Structure - Backend

```
cmd/
  api/
  worker/
internal/
  domain/
  http/
    handlers/
    middleware/
  services/
  repositories/
  ai/
```

```
  schema/
  renderer/
  billing/
  platform/
    postgres/
    redis/
    logger/
migrations/
sqlc.yaml
```

20.3 Production Engineering Critique

Jangan Dibuat Asal

Kesalahan terbesar produk seperti ini adalah membiarkan AI membuat React code bebas. Itu terlihat cepat di demo, tapi gagal saat scale: style tidak konsisten, code tidak maintainable, sulit dites, dan raw generated HTML bisa menjadi security risk. MVP yang benar adalah schema-first dengan renderer terkontrol.

20.4 MVP Build Order

34. Bangun design system dan 10-12 component registry inti.
35. Bangun schema validator dan domain presets.
36. Bangun renderer dari schema ke preview.
37. Baru integrasikan AI untuk generate schema.
38. Tambahkan versioning dan copy code.
39. Tambahkan credit ledger dan billing.
40. Tambahkan refinement dan template gallery.

Appendix A - Minimum MVP Component Registry

Component	Use Case	Required Props
StatsGrid	Metric cards	items[label,value,trend,icon]
ChartPanel	Analytics visual	title, chartType, datasetPreset
DataTable	CRUD/list page	title, columns, rowsPreset, actions
FilterToolbar	Search/filter/action	searchPlaceholder, filters, primaryAction
FormSection	Create/edit form	title, fields, submitLabel
ProfileSummary	Detail/profile page	entity, fields, actions
ActivityTimeline	Recent events	title, items
TabbedContent	Detail sections	tabs
ActionFooter	Form action	primaryLabel, secondaryLabel

Appendix B - Production Readiness Checklist

- OpenAPI spec tersedia dan digunakan oleh FE typed client.
- Migration database sudah idempotent dan reversible.
- Worker generation memiliki retry, timeout, dan dead-letter handling.
- Credit ledger memiliki test untuk concurrent usage.
- Preview iframe sandbox policy sudah diterapkan.
- Schema validator punya test untuk invalid AI output.
- RBAC dan ownership test tersedia.
- Logging tidak menyimpan raw password, token, atau payment secret.
- Admin console tidak membuka prompt private user tanpa alasan operasional jelas.
- Backup dan restore database sudah direncanakan sebelum public beta.